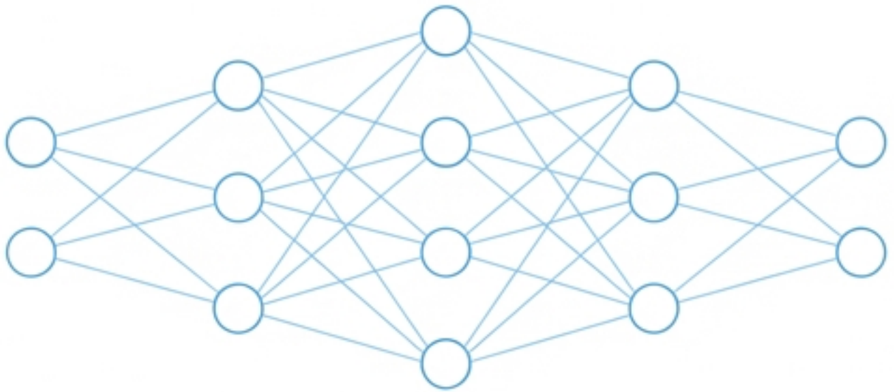
Build Your First End-to-End Gen AI App



- A practical guide to building a real-world financial research tool.
- Learn to harness the power of Large Language Models (LLMs) on your own data.
- Technologies: LangChain, OpenAI, and Streamlit.
- From a real-life industry use case to a working application.

NotebookLM

The Analyst's Dilemma: Too Much Information



- Meet Peter Pandey, an equity research analyst. His job is to read hundreds of news articles and reports daily.
- **The problem:** Manually finding specific data points is tedious, slow, and prone to error.
- Answers are often spread across multiple documents, requiring complex synthesis.
- The goal: Build an **AI-powered tool** to automate this research, turning an overwhelming task into a **simple query**.

NotebookLM

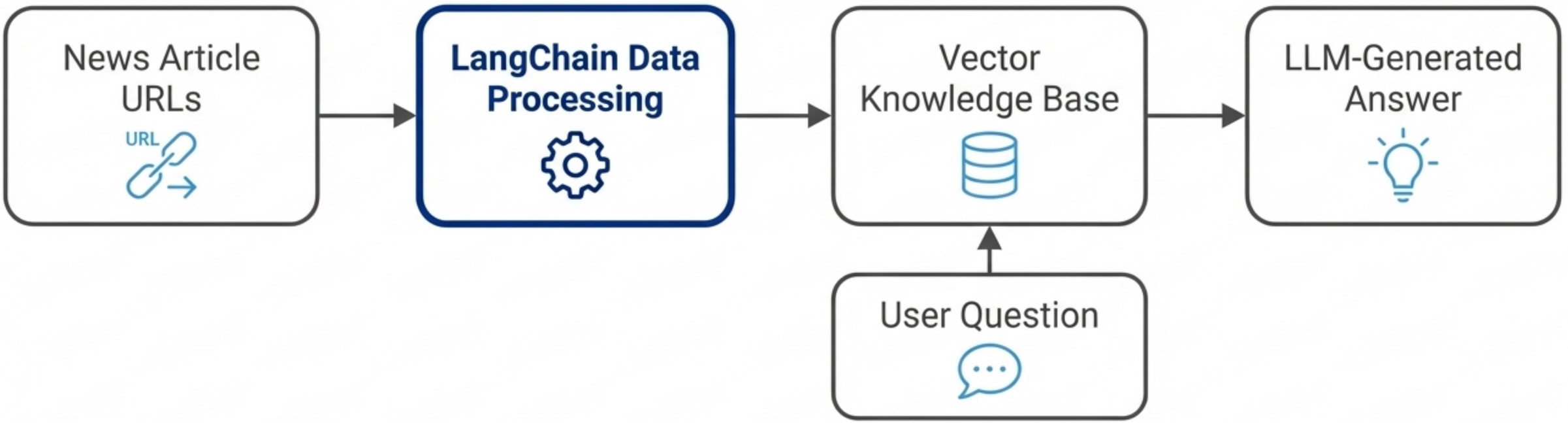
Our Mission: A Custom AI Research Assistant

Standard ChatGPT	Our Custom Tool
Tedious copy-pasting required	Handles private data
Strict word limits	No word limits
Cannot synthesize across sources	Aggregates knowledge across many sources

- We will build a tool that ingests data from a list of news article URLs.
- Users can then ask questions in plain English and get answers sourced directly from those articles.
- Standard chatbots fail because of tedious copy-pasting, strict word limits, and an inability to synthesize knowledge across multiple long documents.
- We need a system that can handle a large, private library of documents.

NotebookLM

The Technical Blueprint



- We ingest documents from URLs.
- We process this text into a format the AI can understand: a Vector Knowledge Base.
- When a user asks a question, we search this base for the most relevant information.
- We send *only* the relevant info and the question to the LLM for a precise, cost-effective answer.

NotebookLM

The Core Insight: Work Smarter, Not Harder

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo.

Relevant Chunk

The company reported strong Q3 EPS of $1.45, exceeding analyst expectations of $1.20. Revenue grew by 15% to $5.2 billion, driven by strong software sales.

Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt. Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet, consectetur, adipisci velit, sed quia non numquam eius modi tempora incidunt ut labore et dolore magnam aliquam quaerat voluptatem.

Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae consequatur, vel illum qui dolorem eum fugiat quo voluptas nulla pariatur?



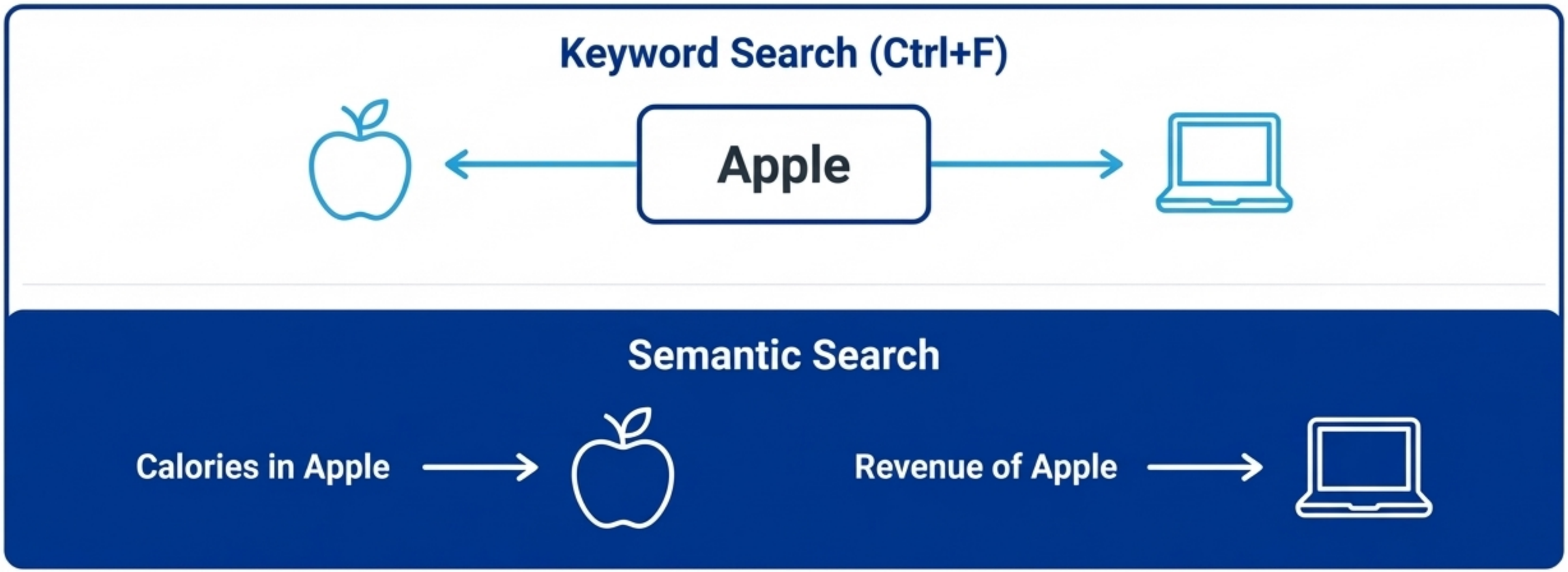
What was the Q3 EPS?



- Sending entire articles to an LLM via API is expensive and inefficient. The cost is based on the amount of text (tokens) you send.
- The answer to a specific question is usually in one or two small paragraphs.
- Our goal is to intelligently find these small, relevant “chunks” of text *before* calling the LLM.
- This saves money, reduces processing time, and often leads to more accurate answers.

NotebookLM

The Magic of Semantic Search



- Keyword search doesn't understand context or meaning.
- Semantic search understands the *intent* behind the words.
- It knows 'price of a car' is related to 'automotive sales figures' even if the exact words don't match.
- This is the technology that lets us find the most relevant chunks for a user's question.

NotebookLM

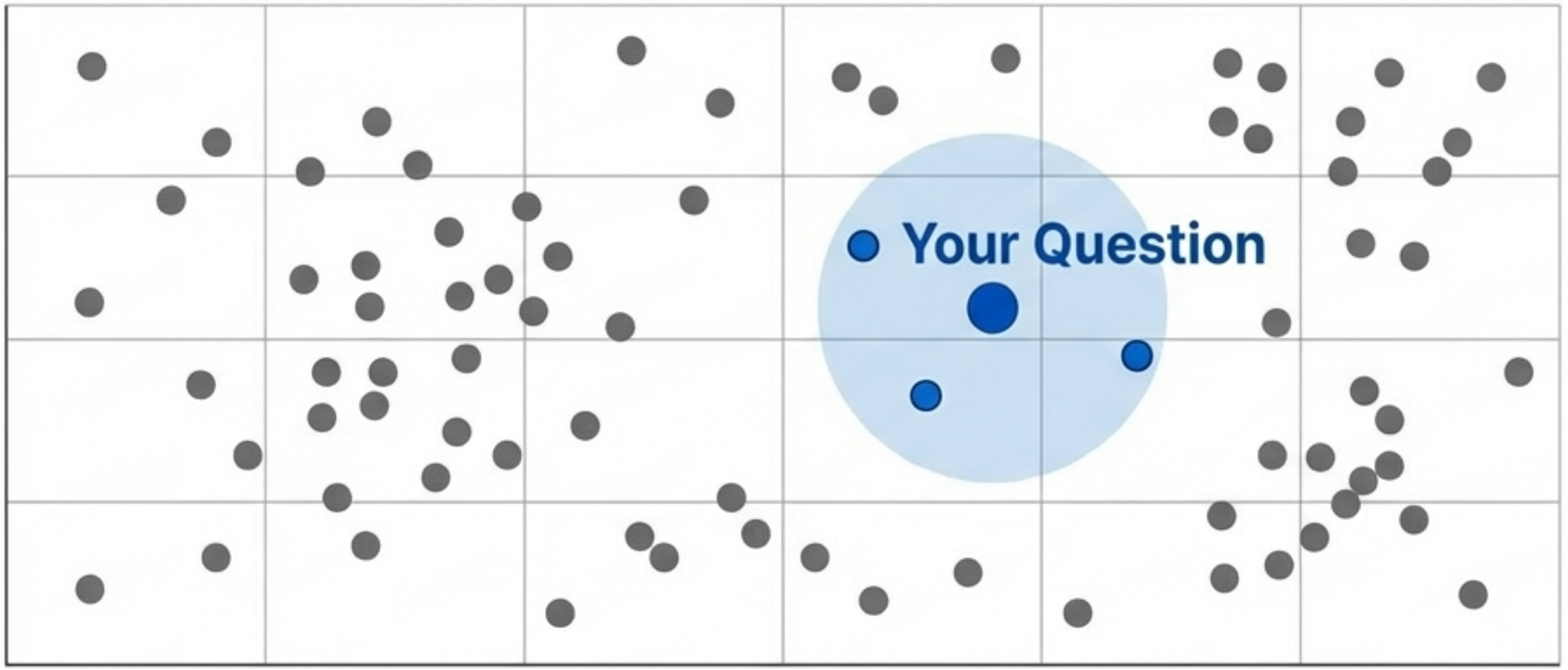
Embeddings: The Language of AI



- To perform semantic search, we must convert text into numbers.
- An “embedding” is a process that turns a piece of text into a list of numbers called a vector.
- This vector represents the text’s semantic meaning in a high-dimensional space.
- Texts with similar meanings will have mathematically similar vectors (i.e., they will be “close” to each other in that space).

NotebookLM

Vector Databases: A GPS for Meaning



- A vector database stores all the number vectors (embeddings) from our documents.
- When you ask a question, we convert your question into a vector too.
- The database then performs a similarity search to find the vectors that are mathematically ‘closest’ to your question’s vector.
- This is how we retrieve the most semantically relevant chunks, instantly.

NotebookLM

Our Toolkit: The Tech Stack



- **LangChain:** The framework that connects and coordinates all the different components.
- **OpenAI**: Provides the powerful LLM for final answers and the model for creating embeddings.
- **FAISS**: A super-fast library for similarity search. It will act as our in-memory vector database.
- **Streamlit**: A simple Python library to build and share our web app's user interface.

NotebookLM

Step 1: Loading Data from the Web

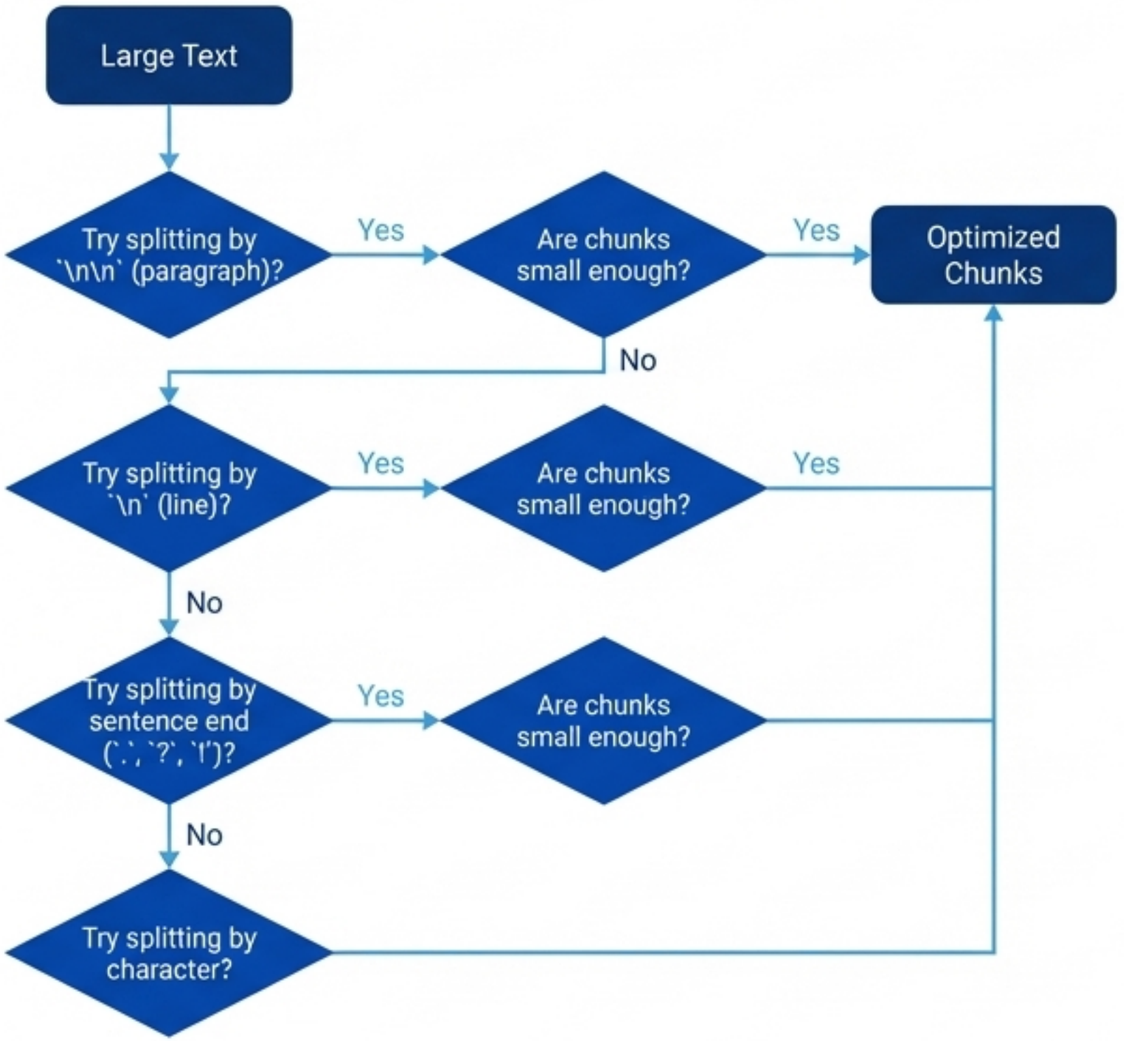


- LangChain provides 'Document Loaders' to easily ingest data from various sources.
- We use '`UnstructuredURLLoader`' to read content directly from news article web pages.
- This loader automatically extracts the main text content from the messy HTML, saving us from writing a web scraper.

```
from langchain.document_loaders import UnstructuredURLLoader
urls = [ "https://www.moneycontrol.com/news/business/story-1.html",
         "https://www.moneycontrol.com/news/business/story-2.html"
]
loader = UnstructuredURLLoader(urls=urls)
data = loader.load()
```

NotebookLM

Step 2: Splitting Text into Smart Chunks



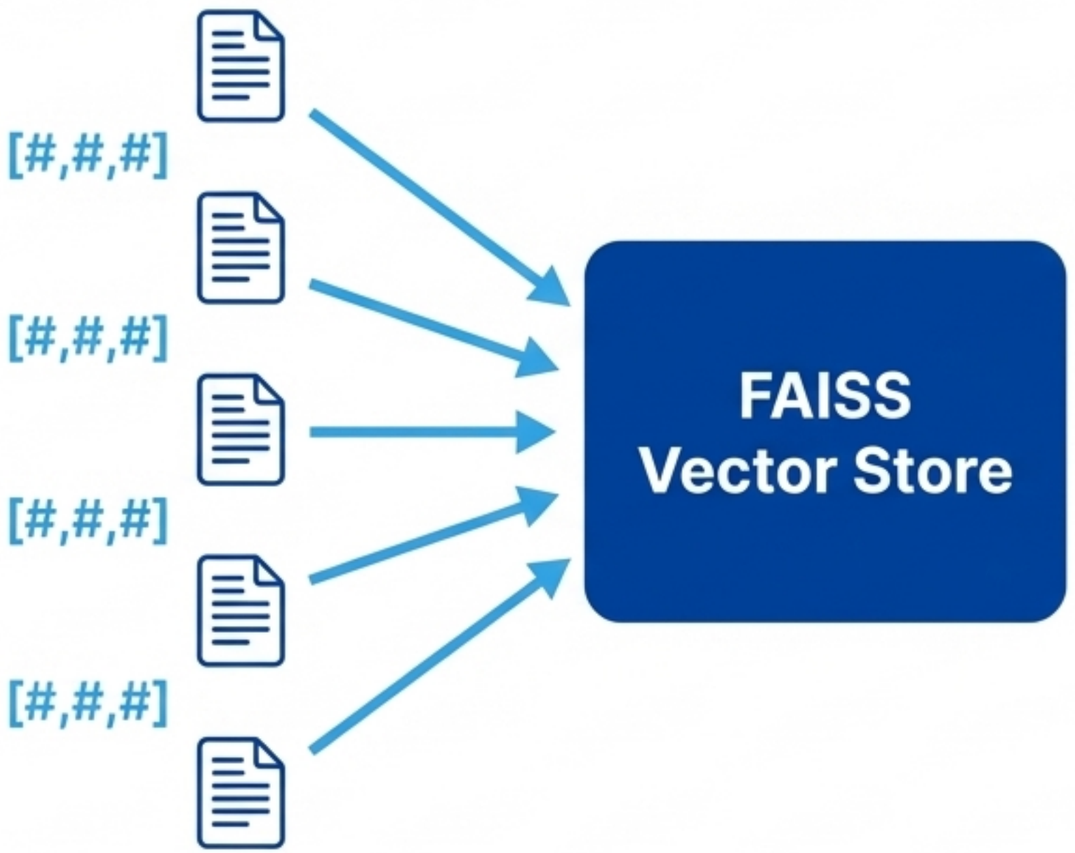
- LLMs have a context window limit; we can't feed them a whole document at once. We must split our data into smaller chunks.
- A simple split might awkwardly cut words or sentences in half.
- `RecursiveCharacterTextSplitter` intelligently tries to split along logical boundaries (paragraphs, then lines, then sentences).
- We also add a small 'overlap' between chunks to preserve context across splits.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
docs = text_splitter.split_documents(data)
```

NotebookLM

Step 3 & 4: Creating the Searchable Memory



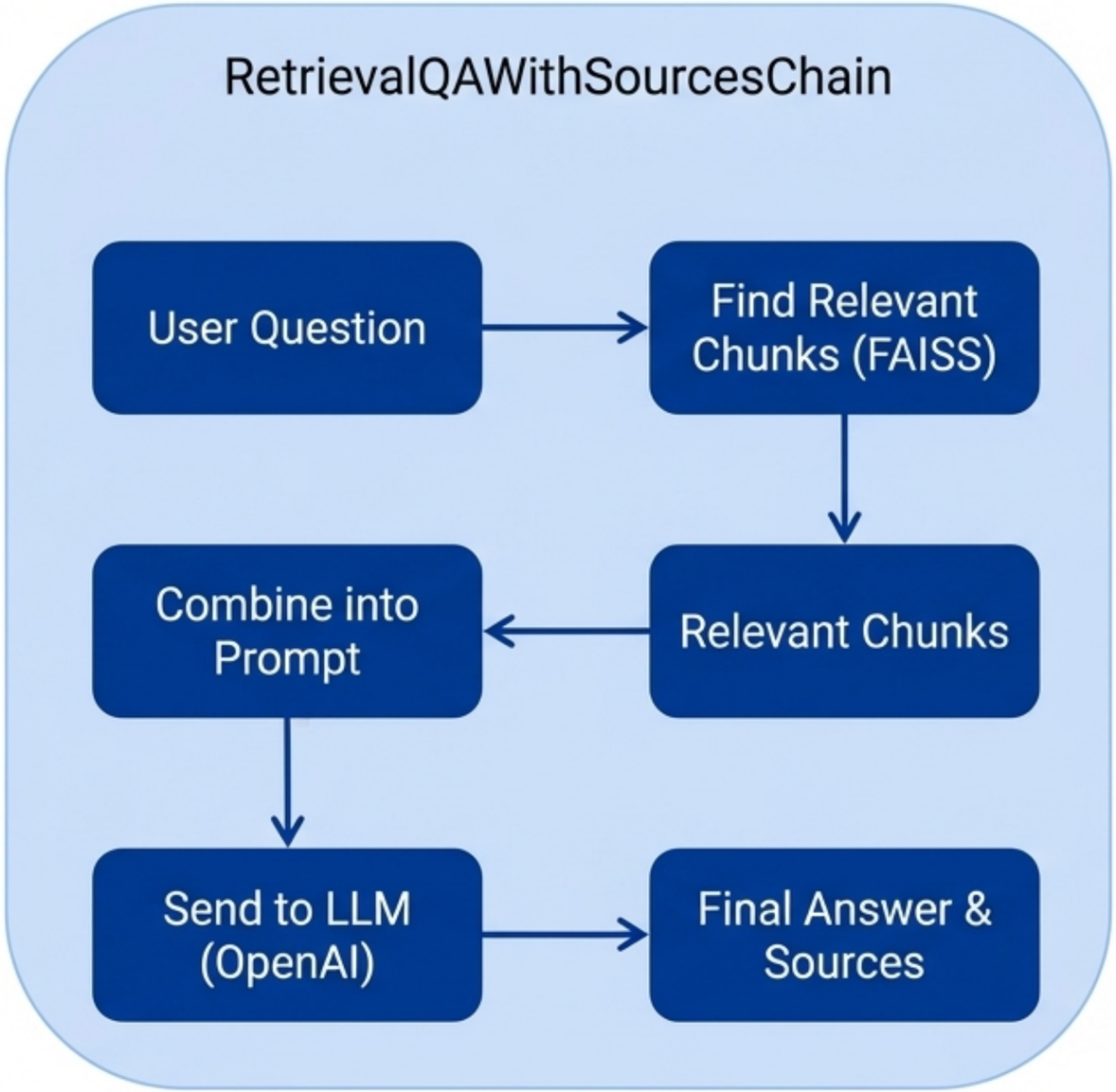
- Now we convert our text chunks into numerical vectors using OpenAI's embedding models.
- FAISS (Facebook AI Similarity Search) is a library for hyper-efficient vector search. We'll use it as our in-memory vector database.
- LangChain combines these steps: the `FAISS.from_documents` command automatically creates embeddings for each document and builds the FAISS index in one go.

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS

embeddings = OpenAIEmbeddings()
# This single line creates embeddings for all docs and stores them
vectorstore = FAISS.from_documents(docs, embeddings)
```

NotebookLM

Step 5: The Question-Answering Engine



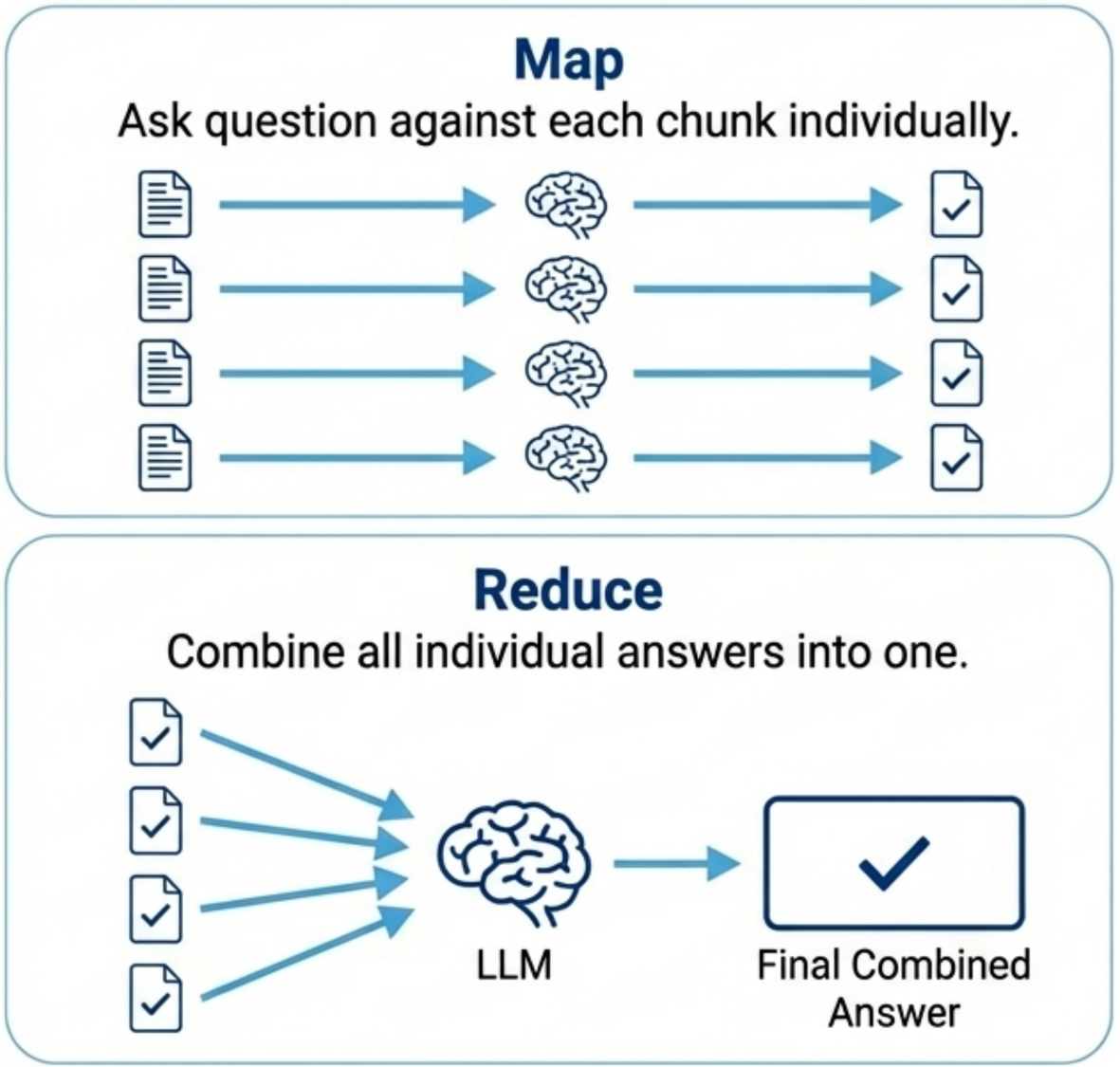
- The `RetrievalQAWithSourcesChain` is the heart of our application.
- It automates the entire process: taking a query, retrieving relevant documents from the vector store, and passing them to the LLM.
- It 'chains' all our previous steps together into a single, callable object.
- Crucially, it also keeps track of which source document each chunk came from, allowing for citations.

```
from langchain.chains import RetrievalQAWithSourcesChain
from langchain.llms import OpenAI

chain = RetrievalQAWithSourcesChain.from_llm(
    llm=OpenAI(temperature=0),
    retriever=vectorstore.as_retriever()
)
```

NotebookLM

Inside the Chain: The ‘MapReduce’ Strategy



- What if our search returns too many chunks to fit into the LLM’s context window?
- The "MapReduce" strategy solves this by breaking the problem down.
- **Map:** First, it asks the LLM the same question against *each chunk individually*.
- **Reduce:** Then, it takes all those individual answers and uses the LLM one last time to synthesize them into a final, coherent response.

NotebookLM

Building the App with Streamlit

- Streamlit lets us build interactive web apps using only Python, which is perfect for rapid prototyping.
- `st.title()` creates the main header for our application.
- `st.sidebar.text_input()` creates input boxes in the left sidebar for our article URLs.
- `st.sidebar.button()` creates the 'Process URLs' button that will trigger our entire LangChain data pipeline.
- `st.text_input()` creates the main question box for the user.

```python
import streamlit as st

st.title("News Research Tool 📈")
st.sidebar.title("News Article URLs")

urls = []
# Code to create sidebar inputs and append to urls list

process_url_clicked = st.sidebar.button("Process URLs")

if process_url_clicked:
    # Logic from slides 10-12 goes here
    ...

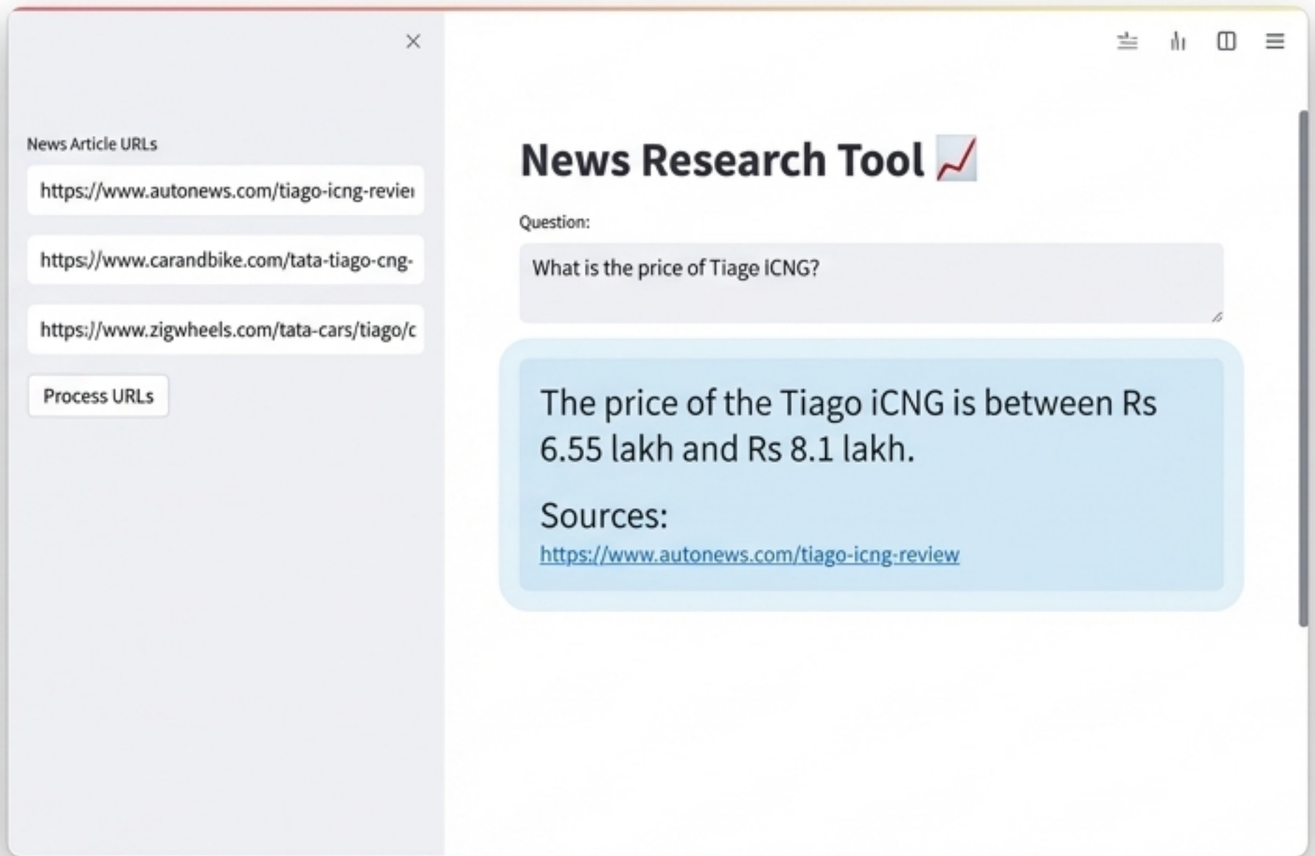
query = st.text_input("Question: ")
if query:
    # Logic from slide 13 goes here
    ...
```



NotebookLM

The Final Product in Action

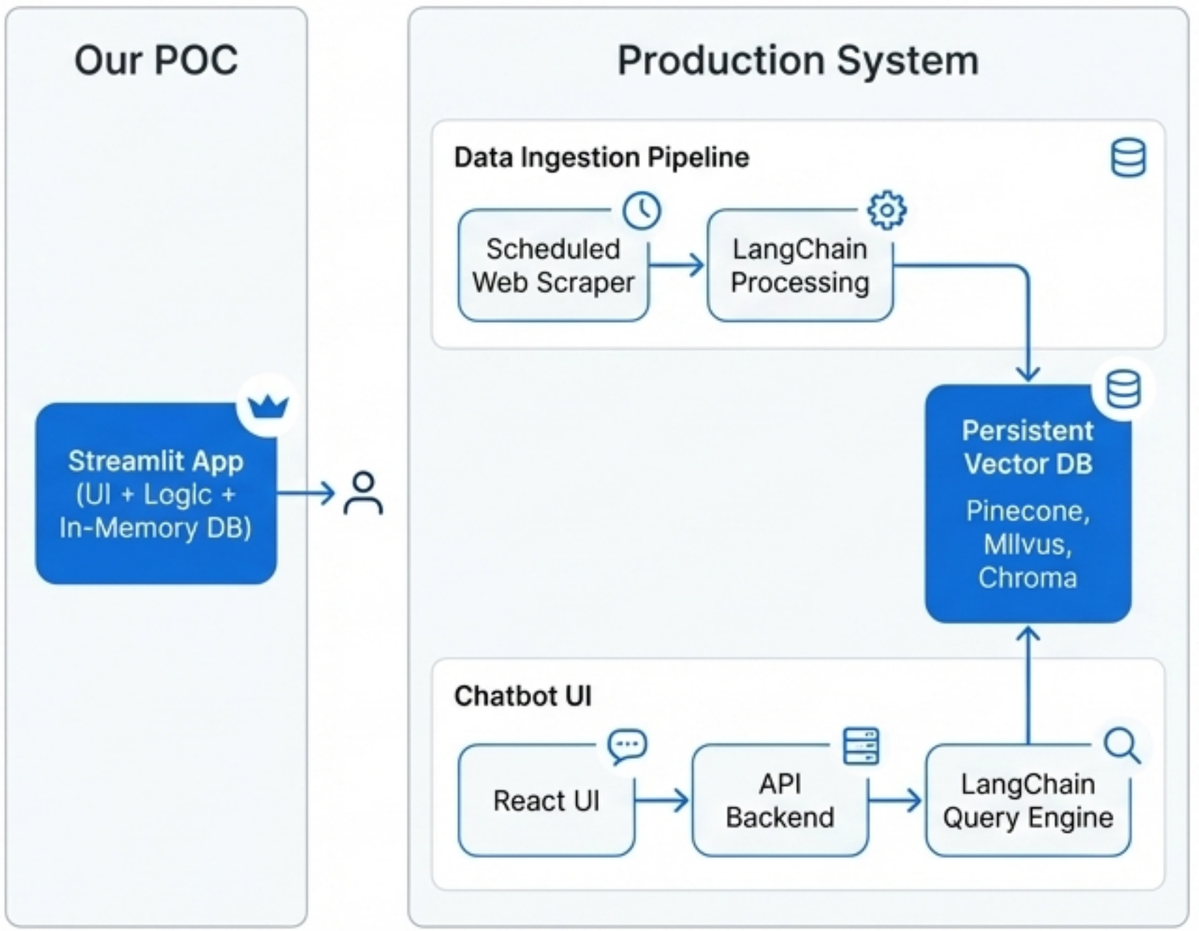
- Our tool successfully ingests raw data from the web.
- It creates a searchable, semantic knowledge base in memory.
- It accurately answers specific questions based *only* on the provided sources.
- It provides verifiable references, making it a trustworthy and practical research assistant.



NotebookLM

Beyond the Prototype: A Production Architecture

- What we built is a powerful Proof of Concept (POC).
- A full production system would separate data ingestion from the user-facing app.
- The “Data Ingestion” pipeline would run automatically (e.g., every hour) to keep the knowledge base updated with new articles.
- The “Chatbot” would be a robust UI that queries this constantly-updated, persistent vector database.



NotebookLM

Key Takeaways & Your Journey Starts Now



Experiment



Build



Share

- You learned **Retrieval Augmented Generation (RAG):** The core pattern of retrieving relevant data to augment an LLM prompt.
- **Experiment**: Try different document loaders (PDFs, CSVs), other embedding models, or a different vector store.
- **Build**: Adapt this code for a new use case. Analyze your own documents, research papers, or personal notes.
- The complete code is available for you to download, run, and modify.

```
# Get the code and start building!
git clone <repository_url>
cd <repository_folder>
pip install -r requirements.txt
streamlit run main.py
```

NotebookLM